

Multi-Class Classification

Erisa Terolli

Slide Credits to Shusen Wang

Multi-Class Classification

Tasks

Methods

Algorithms

Multi-Class Classification

Example 1: face recognition.

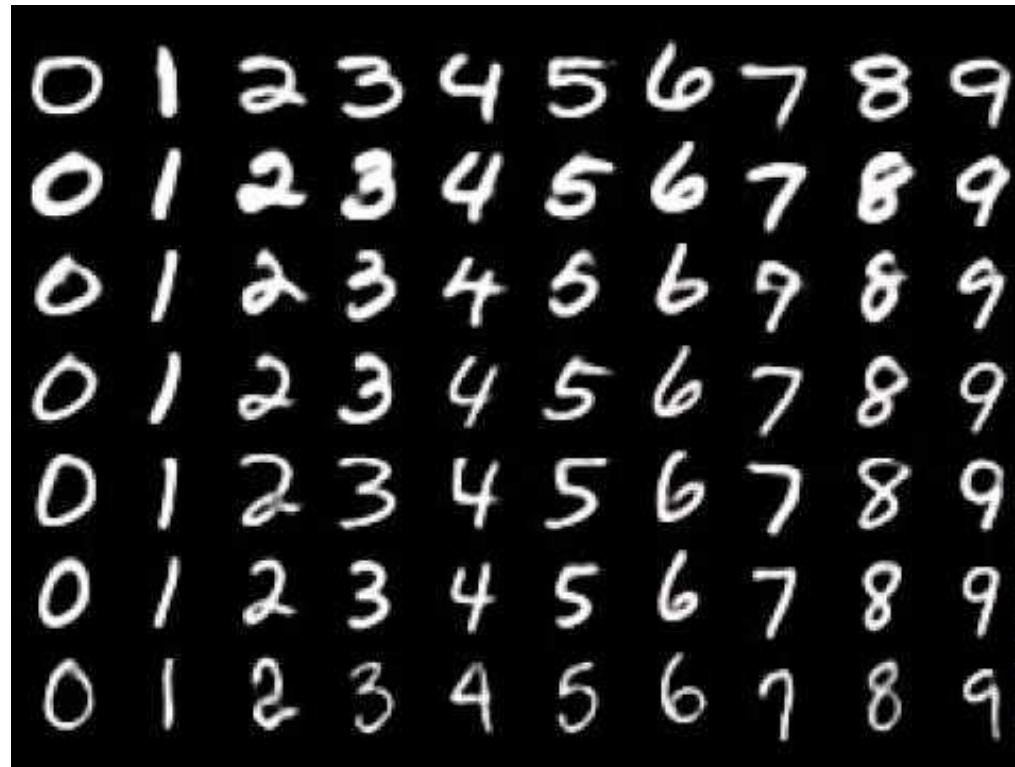
- #classes = #people



Multi-Class Classification

Example 2: hand-written digit recognition.

- #classes = 10



Can we use linear regression?

Looks like “3” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 3



$\mathbf{x}$

Looks like “8” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 8

Can we use linear regression?



$\mathbf{x}$

Looks like “3” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 3



$f = \mathbf{w}^T \mathbf{x}$ is 4.835



Looks like “8” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 8

Can we use linear regression?



$\mathbf{x}$

Looks like “3” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 3



$f = \mathbf{w}^T \mathbf{x}$ is 4.835

rounding

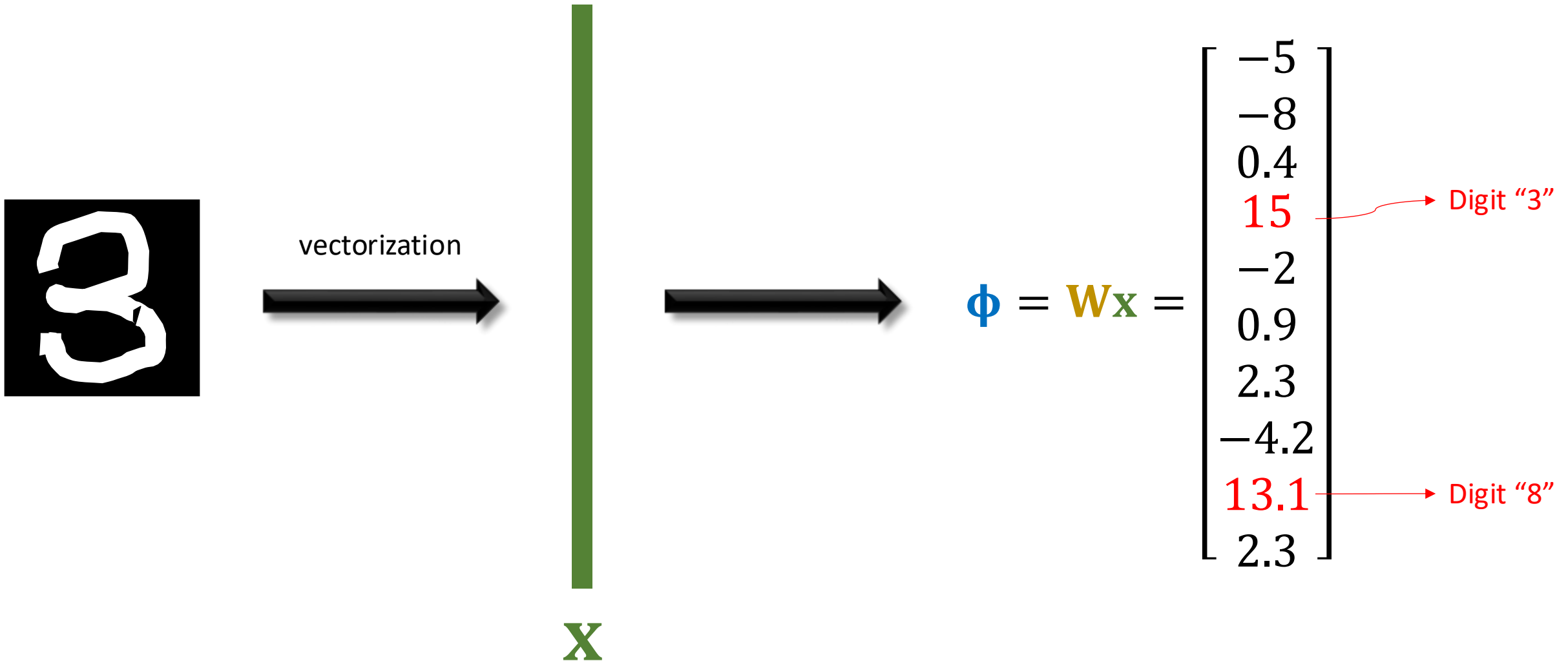


Linear regression believes $\mathbf{x}$ is “5”



Looks like “8” $\rightarrow f = \mathbf{w}^T \mathbf{x}$ is close to 8

The Right Approach



Preliminaries

One-Hot Encoding

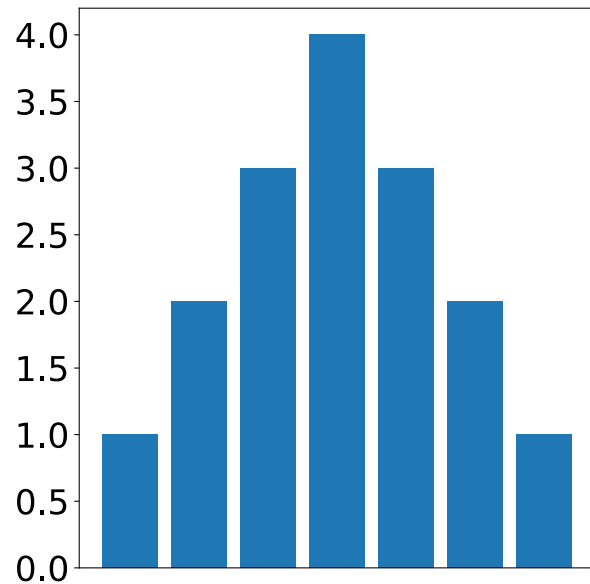
- #Class = 10 (e.g., in digit recognition).
- One-hot encode of $y = 3$:

$$\mathbf{y} = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]^T \in \{1, 0\}^{10}$$

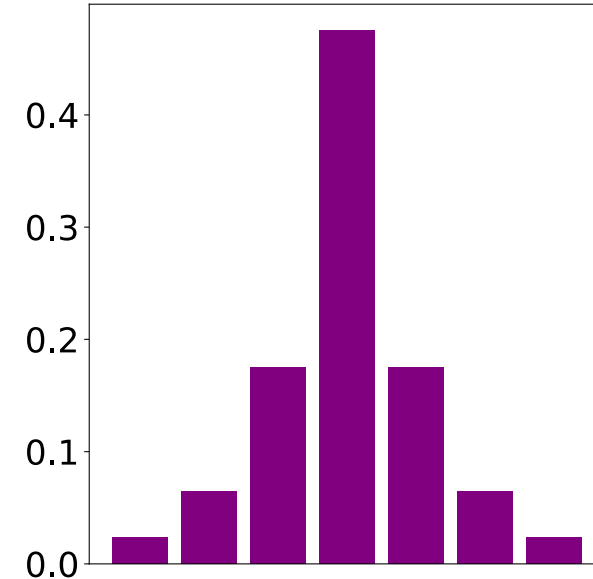
Softmax Function

- $\phi \in \mathbb{R}^K$
- $\mathbf{p} = \text{SoftMax}(\phi) \in \mathbb{R}^K$; its entries are

$$p_k = \frac{\exp(\phi_k)}{\sum_{j=1}^K \exp(\phi_j)}, \text{ for } k = 1, \dots, K.$$



SoftMax



Cross-Entropy

- The vectors $\mathbf{y}$ and $\mathbf{p}$ are K -dim vectors with nonnegative entries.

$$y_1 + \cdots + y_K = 1 \quad \text{and} \quad p_1 + \cdots + p_K = 1.$$

- Cross-entropy between $\mathbf{y}$ and $\mathbf{p}$:

$$H(\mathbf{y}, \mathbf{p}) = -\sum_{l=1}^K y_l \log p_l .$$

Cross-Entropy

- The vectors $\mathbf{y}$ and $\mathbf{p}$ are K -dim vectors with nonnegative entries.

$$y_1 + \cdots + y_K = 1 \quad \text{and} \quad p_1 + \cdots + p_K = 1.$$

- Cross-entropy between $\mathbf{y}$ and $\mathbf{p}$:

$$H(\mathbf{y}, \mathbf{p}) = -\sum_{l=1}^K y_l \log p_l.$$

- Cross-entropy measures the dissimilarity between $\mathbf{y}$ and $\mathbf{p}$.
- When used as loss function, $H(\mathbf{y}, \mathbf{p})$ can be replaced by

$$\|\mathbf{y} - \mathbf{p}\|_2^2 \quad \text{or} \quad \|\mathbf{y} - \mathbf{p}\|_1.$$

Softmax Classifier: Model Formulation

Tasks

Methods

Algorithms

Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes

Remark: If the given labels are scalars $y_1, \dots, y_n \in \{0, 1, \dots, K - 1\}$, turn them to K -dim vectors $\mathbf{y}_1, \dots, \mathbf{y}_n \in \{0, 1\}^K$ using one-hot encoding.

Example: One-hot encode of $y_i = 3$ (where $K=10$):

$$\mathbf{y}_i = [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]^T \in \{0, 1\}^{10}$$

Softmax Classifier

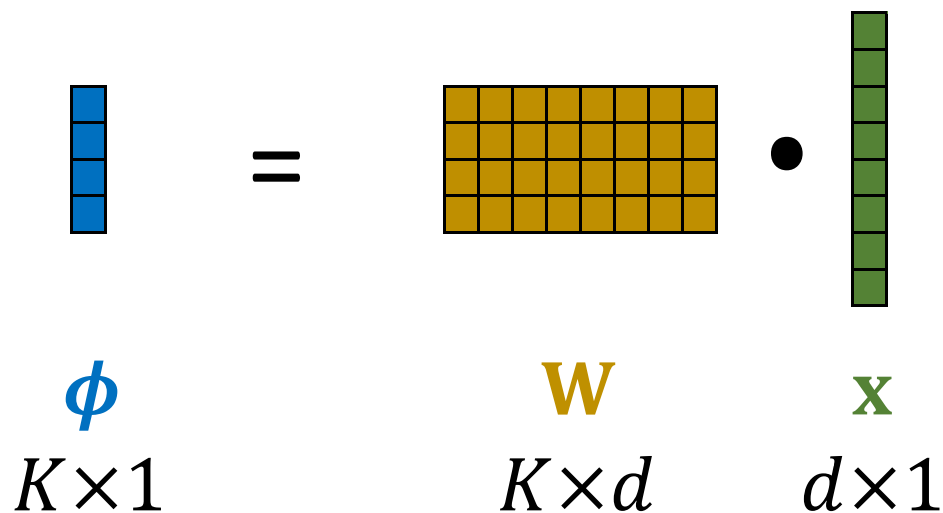
Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes

- $\phi = \mathbf{W}\mathbf{x} \in \mathbb{R}^K$ Here, $\mathbf{x}$ is one of $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$


$$\begin{matrix} \begin{matrix} \text{blue column} \\ \phi \\ K \times 1 \end{matrix} & = & \begin{matrix} \text{yellow matrix} \\ \mathbf{W} \\ K \times d \end{matrix} \cdot \begin{matrix} \text{green column} \\ \mathbf{x} \\ d \times 1 \end{matrix} \end{matrix}$$

Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes

- $\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K$
- ϕ_k (the k -th entry of $\boldsymbol{\phi}$) indicates how likely $\mathbf{x}$ is in the k -th class.

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} = \begin{bmatrix} -5 \\ -8 \\ 0.4 \\ 15 \\ -2 \\ 0.9 \\ 2.3 \\ -4.2 \\ 13.1 \\ 2.3 \end{bmatrix}$$

→ Digit "3"

→ Digit "8"

Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes

- $\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K$
- ϕ_k (the k -th entry of $\boldsymbol{\phi}$) indicates how likely $\mathbf{x}$ is in the k -th class.
- Softmax function: $p_k = \frac{\exp(\phi_k)}{\sum_{j=1}^K \exp(\phi_j)}$.

- $p_1 + \dots + p_K = 1$.
- Thus $\mathbf{p} = [p_1, \dots, p_K] \in \mathbb{R}^K$ is a distribution.

Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes

- $\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K$
- ϕ_k (the k -th entry of $\boldsymbol{\phi}$) indicates how likely $\mathbf{x}$ is in the k -th class.
- Softmax function: $p_k = \frac{\exp(\phi_k)}{\sum_{j=1}^K \exp(\phi_j)}$.
- Cross-entropy loss: $H(\mathbf{y}, \mathbf{p}) = -\sum_{k=1}^K y_k \log p_k$.

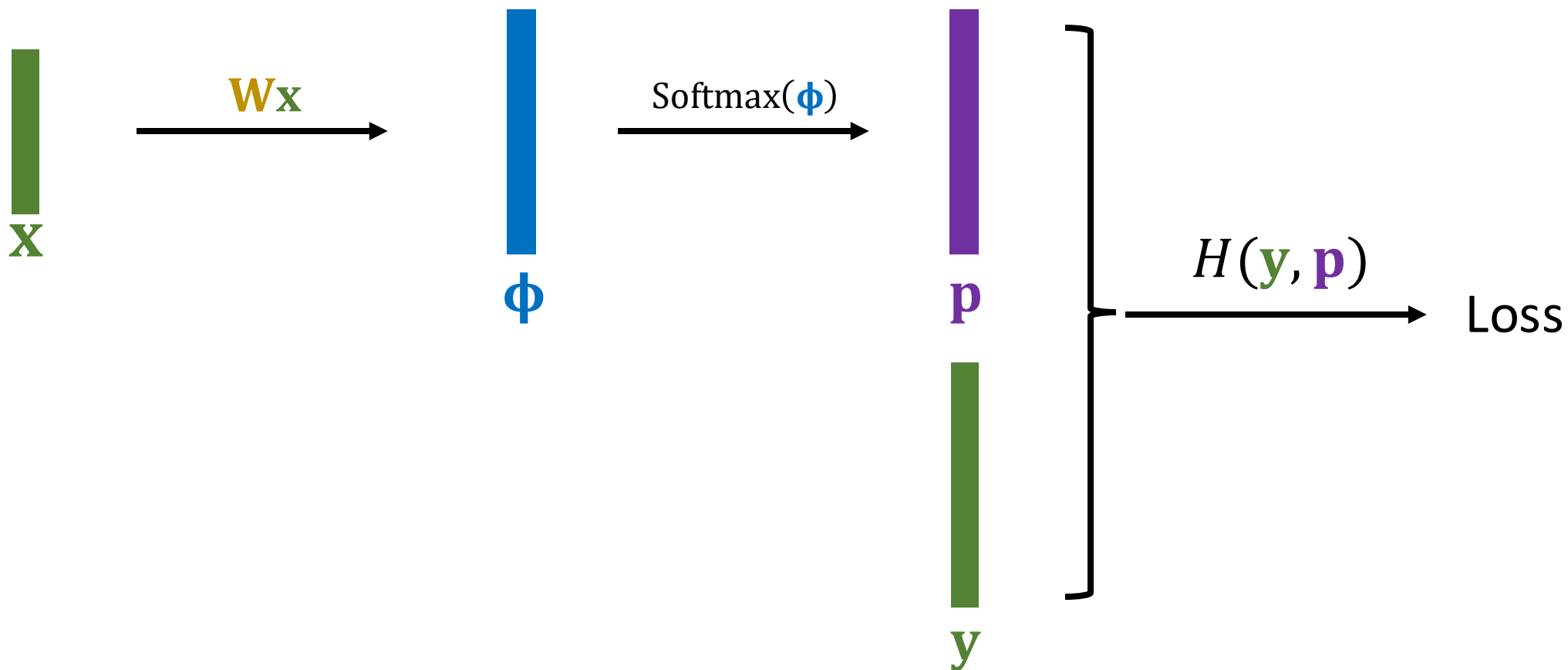
Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

n : #samples

d : #features

K : #classes



Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

• Softmax classifier: $\min_{\mathbf{W}} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$

$$\boldsymbol{\phi}_i = \mathbf{W}\mathbf{x}_i \in \mathbb{R}^K, \quad \mathbf{p}_i = \text{SoftMax}(\boldsymbol{\phi}_i), \quad \text{and} \quad H(\mathbf{y}_i, \mathbf{p}_i) = -\sum_{k=1}^K y_{i,k} \log(p_{i,k}).$$



Softmax function



Cross-entropy loss

Softmax Classifier

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

• Softmax classifier: $\min_{\mathbf{W}} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$

$$\boldsymbol{\phi}_i = \mathbf{W}\mathbf{x}_i \in \mathbb{R}^K, \quad \mathbf{p}_i = \text{SoftMax}(\boldsymbol{\phi}_i), \quad \text{and} \quad H(\mathbf{y}_i, \mathbf{p}_i) = -\sum_{k=1}^K y_{i,k} \log(p_{i,k}).$$

- The role of minimizing $H(\mathbf{y}_i, \mathbf{p}_i)$ is making $\mathbf{p}_i$ similar to $\mathbf{y}_i$.
- $H(\mathbf{y}_i, \mathbf{p}_i)$ can be replaced by $\|\mathbf{y}_i - \mathbf{p}_i\|_2^2$.
- In practice, cross-entropy works better.

Softmax Classifier: Algorithm

Tasks

Methods

Algorithms

Softmax Classifier

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K, \quad p_q = \frac{e^{\phi_q}}{\sum_{j=1}^K e^{\phi_j}}, \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \log(p_q).$$

Softmax Classifier

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K, \quad p_q = \frac{e^{\phi_q}}{\sum_{j=1}^K e^{\phi_j}}, \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \log(p_q).$$

- $H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \phi_q + \log\left(\sum_{j=1}^K e^{\phi_j}\right) \cdot \sum_{q=1}^K y_q.$

Softmax Classifier

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K, \quad p_q = \frac{e^{\phi_q}}{\sum_{j=1}^K e^{\phi_j}}, \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \log(p_q).$$

$$\bullet H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \phi_q + \log\left(\sum_{j=1}^K e^{\phi_j}\right) \cdot \sum_{q=1}^K y_q.$$
$$= 1$$

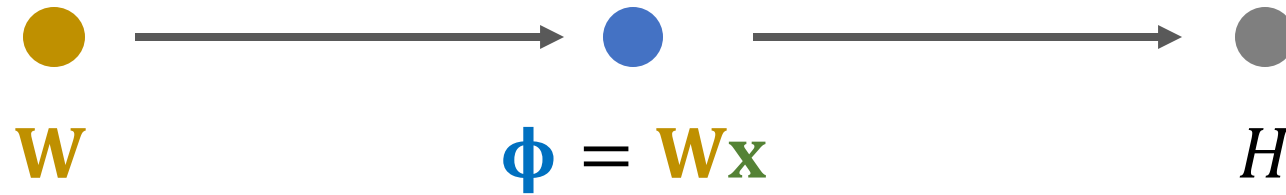
Softmax Classifier

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

$$\bullet H(\mathbf{y}, \mathbf{p}) = -\sum_{q=1}^K y_q \phi_q + \log\left(\sum_{j=1}^K e^{\phi_j}\right) \cdot \boxed{\sum_{q=1}^K y_q} \\ = 1$$

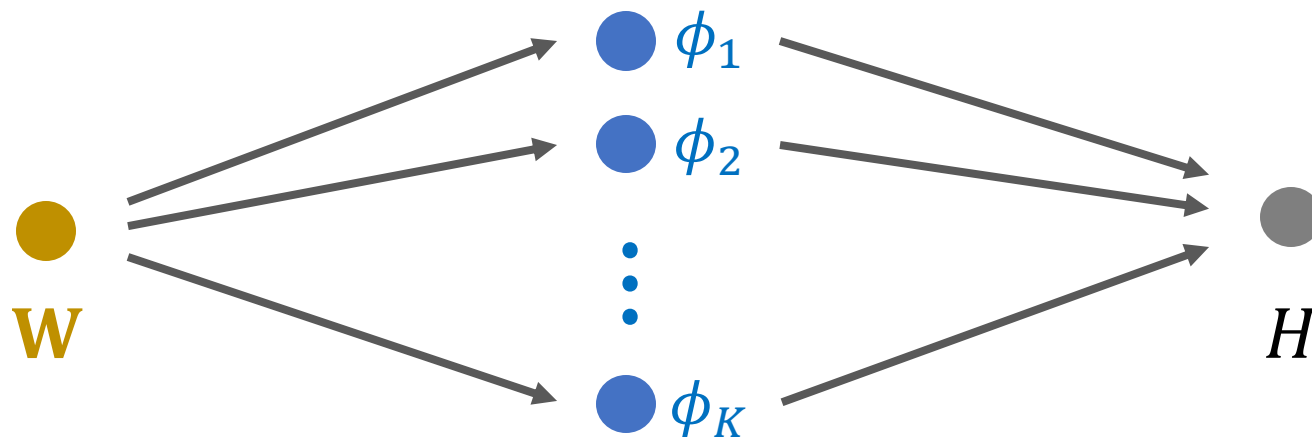
Softmax Classifier

$$\phi = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$



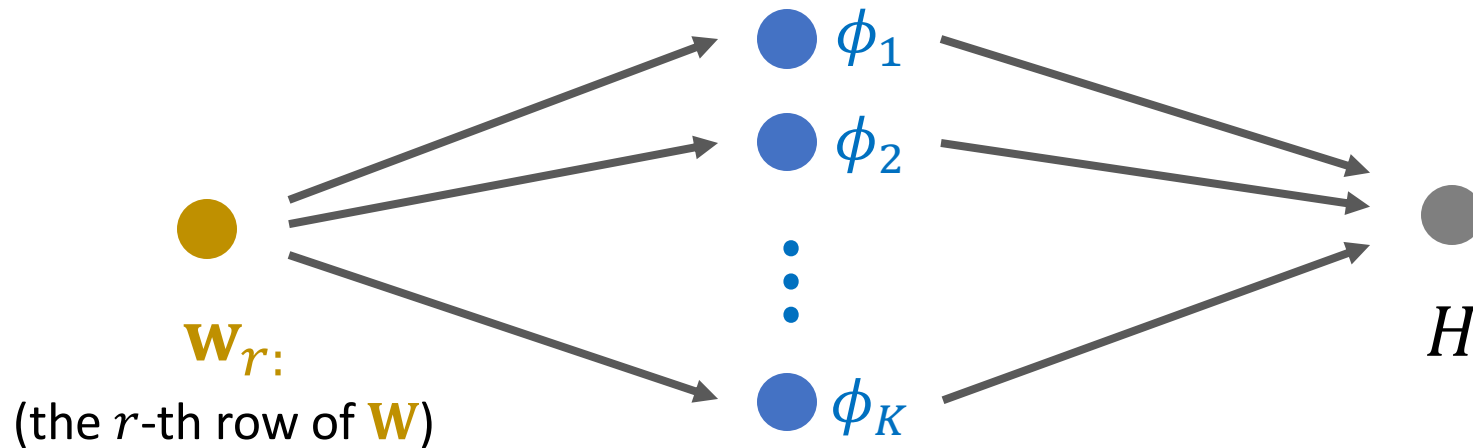
Softmax Classifier

$$\phi = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$



Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

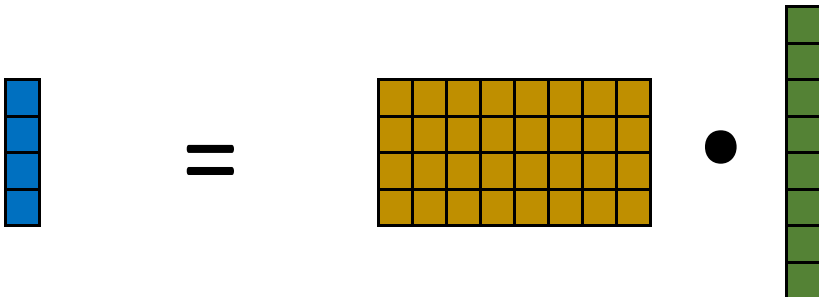


Chain rule:
$$\frac{\partial H}{\partial \mathbf{w}_{r:}} = \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{r:}} \cdot \frac{\partial H}{\partial \phi_q}$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.



$\boldsymbol{\phi}$
 $K \times 1$

$\mathbf{W}$
 $K \times d$

$\mathbf{x}$
 $d \times 1$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.

- $\frac{\partial H}{\partial \mathbf{w}_{1:}} = \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q}$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.

- $$\frac{\partial H}{\partial \mathbf{w}_{1:}} = \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q} = \frac{\partial \phi_1}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_1} + \sum_{q \neq 1} \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q}.$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.

- $$\frac{\partial H}{\partial \mathbf{w}_{1:}} = \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q} = \boxed{\frac{\partial \phi_1}{\partial \mathbf{w}_{1:}}} \cdot \frac{\partial H}{\partial \phi_1} + \sum_{q \neq 1} \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q}.$$

$$= \boxed{\frac{\partial \mathbf{x}^T \mathbf{w}_{1:}}{\partial \mathbf{w}_{1:}}} = \mathbf{x}$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.

- $$\begin{aligned} \frac{\partial H}{\partial \mathbf{w}_{1:}} &= \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q} = \boxed{\frac{\partial \phi_1}{\partial \mathbf{w}_{1:}}} \cdot \frac{\partial H}{\partial \phi_1} + \sum_{q \neq 1} \boxed{\frac{\partial \phi_q}{\partial \mathbf{w}_{1:}}} \cdot \frac{\partial H}{\partial \phi_q} \\ &\quad \boxed{= \frac{\partial \mathbf{x}^T \mathbf{w}_{1:}}{\partial \mathbf{w}_{1:}} = \mathbf{x}} \quad \boxed{= \frac{\partial \mathbf{x}^T \mathbf{w}_{q:}}{\partial \mathbf{w}_{1:}} = \mathbf{0}} \end{aligned}$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\phi_q = \mathbf{x}^T \mathbf{w}_{q:}$.

- $$\frac{\partial H}{\partial \mathbf{w}_{1:}} = \sum_{q=1}^K \frac{\partial \phi_q}{\partial \mathbf{w}_{1:}} \cdot \frac{\partial H}{\partial \phi_q} = \boxed{\frac{\partial \phi_1}{\partial \mathbf{w}_{1:}}} \cdot \frac{\partial H}{\partial \phi_1} + \sum_{q \neq 1} \boxed{\frac{\partial \phi_q}{\partial \mathbf{w}_{1:}}} \cdot \frac{\partial H}{\partial \phi_q}.$$

$$= \frac{\partial \mathbf{x}^T \mathbf{w}_{1:}}{\partial \mathbf{w}_{1:}} = \mathbf{x}$$

$$= \frac{\partial \mathbf{x}^T \mathbf{w}_{q:}}{\partial \mathbf{w}_{1:}} = \mathbf{0}$$

- $$\frac{\partial H}{\partial \mathbf{w}_{1:}} = \mathbf{x} \cdot \frac{\partial H}{\partial \phi_1}.$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

$$\bullet \quad \frac{\partial H}{\partial \mathbf{w}_{q:}} = \mathbf{x} \cdot \frac{\partial H}{\partial \phi_q}.$$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\frac{\partial H}{\partial \mathbf{w}_q} = \mathbf{x} \cdot \frac{\partial H}{\partial \phi_q}.$
- $\frac{\partial H}{\partial \phi_q} = -y_q + \frac{e^{\phi_q}}{\sum_{j=1}^K e^{\phi_j}} = -y_q + p_q.$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

- $\frac{\partial H}{\partial \mathbf{w}_{q:}} = \mathbf{x} \cdot \frac{\partial H}{\partial \phi_q}.$
- $\frac{\partial H}{\partial \phi_q} = -y_q + \frac{e^{\phi_q}}{\sum_{j=1}^K e^{\phi_j}} = -y_q + p_q.$
- $\Rightarrow \frac{\partial H}{\partial \mathbf{w}_{q:}} = \mathbf{x} \cdot (-y_q + p_q)$

Gradient

$$\boldsymbol{\phi} = \mathbf{W}\mathbf{x} \in \mathbb{R}^K \quad \text{and} \quad H(\mathbf{y}, \mathbf{p}) = -\sum_{j=1}^K y_j \phi_j + \log\left(\sum_{j=1}^K e^{\phi_j}\right).$$

$$\bullet \frac{\partial H}{\partial \mathbf{w}_{q:}} = (\mathbf{p}_q - \mathbf{y}_q) \cdot \mathbf{x}.$$

$$\bullet \frac{\partial H}{\partial \mathbf{W}} = \begin{bmatrix} \frac{\partial H}{\partial \mathbf{w}_{1:}^T} \\ \frac{\partial H}{\partial \mathbf{w}_{2:}^T} \\ \vdots \\ \frac{\partial H}{\partial \mathbf{w}_{K:}^T} \end{bmatrix} = \begin{bmatrix} (\mathbf{p}_1 - \mathbf{y}_1) \cdot \mathbf{x}^T \\ (\mathbf{p}_2 - \mathbf{y}_2) \cdot \mathbf{x}^T \\ \vdots \\ (\mathbf{p}_K - \mathbf{y}_K) \cdot \mathbf{x}^T \end{bmatrix} = (\mathbf{p} - \mathbf{y}) \cdot \mathbf{x}^T.$$

Stochastic Gradient Descent

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

• Model: $\min_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$

$$\boldsymbol{\phi}_i = \mathbf{W} \mathbf{x}_i \in \mathbb{R}^K, \quad \mathbf{p}_i = \text{SoftMax}(\boldsymbol{\phi}_i), \quad \text{and} \quad H(\mathbf{y}_i, \mathbf{p}_i) = - \sum_{k=1}^K y_{i,k} \log(p_{i,k}).$$

• A (stochastic) gradient: $\mathbf{G}_i = \frac{\partial H(\mathbf{y}_i, \mathbf{p}_i)}{\partial \mathbf{W}} = (\mathbf{p}_i - \mathbf{y}_i) \cdot \mathbf{x}_i^T \in \mathbb{R}^{K \times d}$.

Stochastic Gradient Descent

Input: feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$.

• Model: $\min_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$

$$\boldsymbol{\phi}_i = \mathbf{W} \mathbf{x}_i \in \mathbb{R}^K, \quad \mathbf{p}_i = \text{SoftMax}(\boldsymbol{\phi}_i), \quad \text{and} \quad H(\mathbf{y}_i, \mathbf{p}_i) = - \sum_{k=1}^K y_{i,k} \log(p_{i,k}).$$

• A (stochastic) gradient: $\mathbf{G}_i = \frac{\partial H(\mathbf{y}_i, \mathbf{p}_i)}{\partial \mathbf{W}} = (\mathbf{p}_i - \mathbf{y}_i) \cdot \mathbf{x}_i^T \in \mathbb{R}^{K \times d}$.

SGD Algorithm:

1. Randomly sample i from $\{1, 2, \dots, n\}$.
2. Compute $\mathbf{G}_i$ using $(\mathbf{x}_i, \mathbf{y}_i)$.
3. $\mathbf{W} \leftarrow \mathbf{W} - \alpha \mathbf{G}_i$.

Softmax Classifier: Train and Test

- Train (given feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$)
 - Compute $\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$ by SGD or other algorithms.
- Test (for a sample $\mathbf{x}' \in \mathbb{R}^d$)
 - $\boldsymbol{\phi}' = \mathbf{W}^* \mathbf{x}' \in \mathbb{R}^K$.
 - Return the index of the largest entry of $\boldsymbol{\phi}'$.

Softmax Classifier: Train and Test

- Train (given feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$)
 - Compute $\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$ by SGD or other algorithms.
- Test (for a sample $\mathbf{x}' \in \mathbb{R}^d$)
 - $\boldsymbol{\phi}' = \mathbf{W}^* \mathbf{x}' \in \mathbb{R}^K$.
 - Return the index of the largest entry of $\boldsymbol{\phi}'$.

What is the predicted class?

$$\boldsymbol{\phi}' = \begin{bmatrix} 0.9 \\ -5 \\ 10 \\ -20 \\ 0 \end{bmatrix}$$

Softmax Classifier: Train and Test

- Train (given feature vectors $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ and labels $\mathbf{y}_1, \dots, \mathbf{y}_n \in \mathbb{R}^K$)
 - Compute $\mathbf{W}^* = \underset{\mathbf{W}}{\operatorname{argmin}} \sum_{i=1}^n H(\mathbf{y}_i, \mathbf{p}_i)$ by SGD or other algorithms.
- Test (for a sample $\mathbf{x}' \in \mathbb{R}^d$)
 - $\boldsymbol{\phi}' = \mathbf{W}^* \mathbf{x}' \in \mathbb{R}^K$.
 - Return the index of the largest entry of $\boldsymbol{\phi}'$.

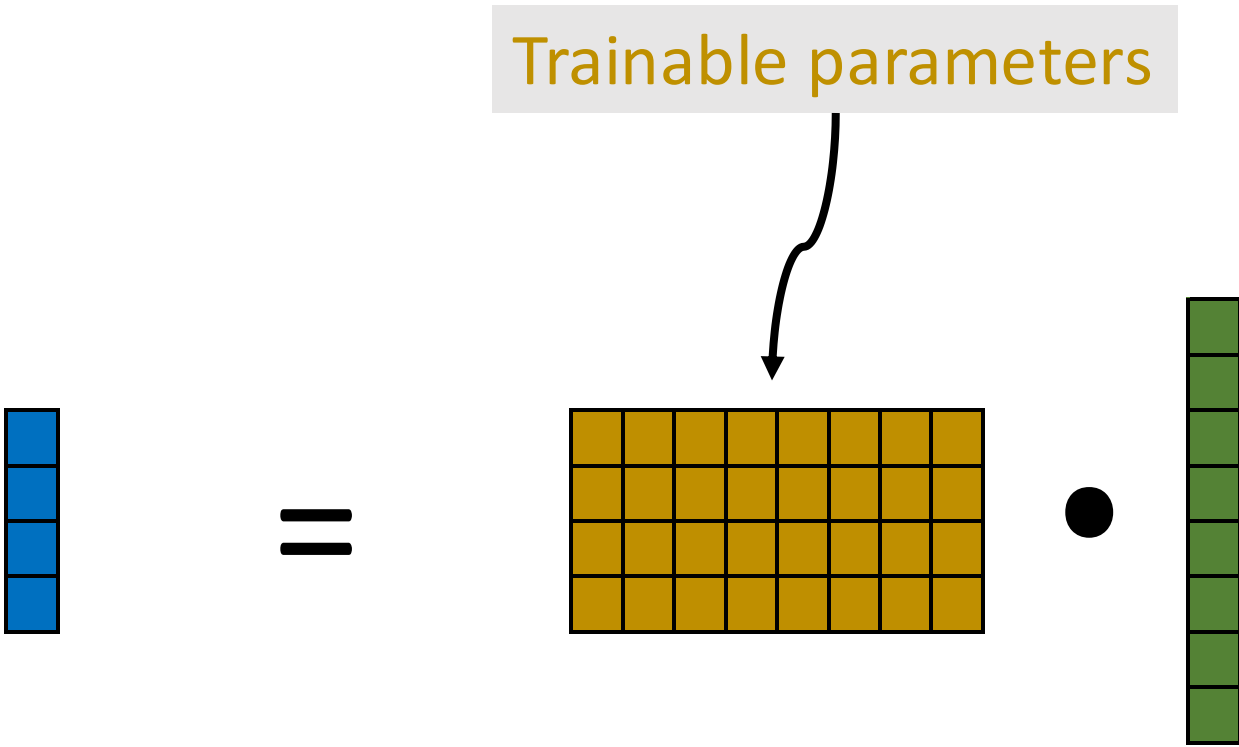
What is the predicted class?

$$\boldsymbol{\phi}' = \begin{bmatrix} 0.9 \\ -5 \\ 10 \\ -20 \\ 0 \end{bmatrix}$$

Limitations of Softmax Classifier

#Parameter v.s. #Classes

Trainable parameters



ϕ

$K \times 1$

$=$

W

$K \times d$

$\cdot$

x

$d \times 1$

#Parameter v.s. #Classes

- Suppose #features = $1K$.
- Suppose #classes = 10 (e.g., digit recognition).
 - $1K \times 10 = 10K$ parameters.
- Suppose #classes = $1K$ (e.g., ImageNet image recognition).
 - $1K \times 1K = 1M$ parameters.
- Suppose #classes = $1M$ (e.g., face recognition).
 - $1K \times 1M = 1G$ parameters → Heavy computation and memory costs.

#Parameter v.s. #Classes

- Suppose $\#features = 1K$.
- Suppose $\#classes = 10$ (e.g., digit recognition).
 - $1K \times 10 = 10K$ parameters.
- Suppose $\#classes = 1K$ (e.g., ImageNet image recognition).
 - $1K \times 1K = 1M$ parameters.
- Suppose $\#classes = 1M$ (e.g., face recognition).
 - $1K \times 1M = 1G$ parameters $\rightarrow$ Heavy computation and memory costs.
- What if $\#classes = 1G$? (E.g., face recognition for all the Chinese citizens.)